

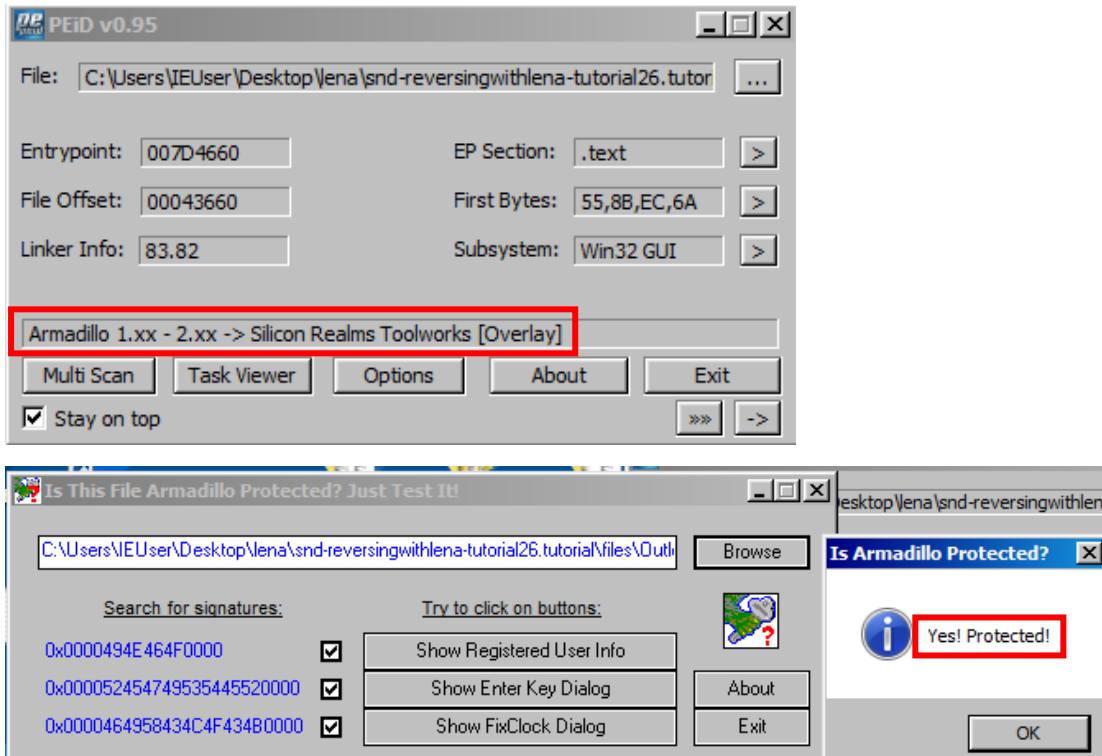
악성코드 분석 보고서

(sand-reversingwithlana-tutorials)

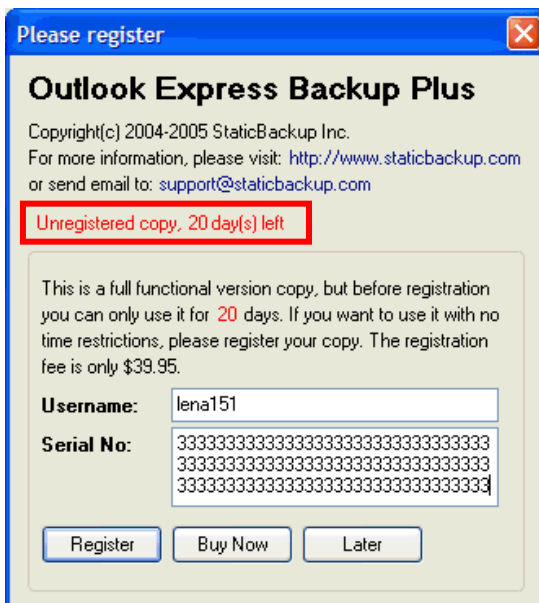
2025.11.10

24, 25와 마찬가지로 실행이 되지 않아 lena영상 및 다른 사용자의 글을 참고하여 작성하였다.

1) OutlookExpressBackup.exe – Loader 사용



PEiD와 IsItArma를 이용해서 확인해본 결과 아르마딜로 패커를 사용한 걸 볼 수 있다.
(버전은 정확하지 않을 수 있다.)



OutlookExpressBackup.exe를 실행했을 때 시리얼이 등록되지 않아서 남은 기간이 뜨는 걸 볼 수 있다.

K Call stack of main thread				
Address	Stack	Procedure / arguments	Called from	Frame

0012CB10	00435CC2	RETURN to OutlookE.00435CC2		
0012CB14	0012CB5C	Pointer to next SEH record		
0012CB18	00435CDB	SE handler		
0012CB1C	0012CB34			
0012CB20	00460AFC	OutlookE.00460AFC		
0012CB24	01F5D244			
0012CB28	01F7EB30	ASCII "tPC"		
0012CB2C	00000000			
0012CB30	010004C6			
0012CB34	0012CB50			
0012CB38	00435C37	RETURN to OutlookE.00435C37 from OutlookE.00435C3C		
0012CB3C	00000000			
0012CB40	FFFFFFFF			
0012CB44	FFFFFFFF			
0012CB48	00000000			

해당 구간에서 Call Stack에 들어가보면 아무것도 없는 걸 볼 수 있는데 이럴 때 시스템 스택 창을 보면 0x00486D2C이 실행되기 전 코드를 볼 수 있다.

00435CBA	8B10	MOV EDX,DWORD PTR DS:[EAX]	
00435CBC	FF92 EC000000	CALL DWORD PTR DS:[EDX+EC]	OutlookE.00486D2C
00435CC2	8945 F8	MOV DWORD PTR SS:[EBP-8],EAX	
00435CC5	33C0	XOR EAX,EAX	
00435CC7	5A	POP EDX	OutlookE.00435CC2
00435CC8	59	POP ECX	OutlookE.00435CC2
00435CC9	59	POP ECX	OutlookE.00435CC2
00435CCA	64:8910	MOV DWORD PTR FS:[EAX],EDX	OutlookE.00435074

0x00435CC2로 넘어가보면 바로 위에 코드에서 EAX의 값을 받는 걸 볼 수 있다.

해당 함수를 호출하지 않으면 이전에 보았던 등록 실패 메시지 박스가 나타나지 않을 것이라고 추정했기 때문에 이전과 마찬가지로 해당 주소를 넘어가는 조건문을 찾아야 한다.

00435C30	55	PUSH EBP	
00435C3D	8BEC	MOV EBP,ESP	
00435C3F	83C4 F4	ADD ESP,-0C	
00435C42	53	PUSH EBX	
00435C43	56	PUSH ESI	
00435C44	66:894D FD	MOV WORD PTR SS:[EBP-3],CX	
00435C48	8855 FF	MOV BYTE PTR SS:[EBP-1],DL	
00435C4B	8B75 0C	MOV ESI,DWORD PTR SS:[EBP+C]	
00435C4E	8B5D 10	MOV EBX,DWORD PTR SS:[EBP+10]	
00435C51	66:8B4D FD	MOV CX,WORD PTR SS:[EBP-3]	
00435C55	8A55 FF	MOV DL,BYTE PTR SS:[EBP-1]	
00435C58	E8 1BFAFFFF	CALL OutlookE.00435678	
00435C5D	00435C74	CALL OutlookE.00435C74	

하지만 조건문을 찾을 수 없어서 0x00435C3C에 BP를 걸고 다시 실행해준다.

K Call stack of main thread				
Address	Stack	Procedure / arguments	Called from	
0012CB38	00435C37	? OutlookE.00435C3C	OutlookE.00435C3C	
0012CB54	0052EF46	? OutlookE.00435C1C	OutlookE.0052EF41	
0012CB58	00000000	Arg1 = 00000000		

다시 실행 후 Call Stack을 보면 0x0052EF41에서 함수를 호출하는 걸 볼 수 있다.

0052EF10	6A 00	PUSH 0	
0052EF12	66:8B0D 98EF5200	MOV CX,WORD PTR DS:[52EF98]	
0052EF18	B2 02	MOV DL,2	
0052EF1B	B8 A4EF5200	MOV EAX,OutlookE.0052EF94	ASCII "Thank you for your register, pl
0052EF20	E8 F76CF0FF	CALL OutlookE.0052EF94	
0052EF25	C783 4C020000 01000000	MOV DWORD PTR EAX,4C020000	
0052EF2F	EB 23	JMP SHORT OutlookE.0052EF31	
0052EF31	6A 00	PUSH 0	
0052EF33	66:8B0D 98EF5200	MOV CX,WORD PTR DS:[52EF98]	
0052EF3A	B2 01	MOV DL,1	
0052EF3C	B8 DCE52000	MOV EAX,OutlookE.0052EF94	ASCII "Sorry, your registration inform
0052EF41	E8 D66CF0FF	CALL OutlookE.00435C1C	
0052EF44	8B83 1C030000	MOV EAX,DWORD PTR DS:[EBX+31C]	

It seems this is where we need to be

해당 함수로 넘어가보면 위에 성공했을 때 나오는 창을 확인할 수 있다. 그럼 이 부분에서 성공창과 실패창을 구분한다는 걸 알 수 있다.

0052EED0	59	POP ECX	OutlookE.00435C37
0052EED8	E8 A4F7FFFF	CALL OutlookE.0052E684	
0052EEE0	84C0	TEST AL,AL	
0052EEE2	74 4D	JE SHORT OutlookE.0052EF31	
0052EEE4	8D55 F0	LEA EDX,DWORD PTR SS:[EBP-10]	
0052EEE7	8B83 40030000	MOV EAX,DWORD PTR DS:[EBX+340]	
0052EEED	E8 F2AAF3FF	CALL OutlookE.004699E4	
0052EEF2	8B45 F0	MOV EAX,DWORD PTR SS:[EBP-10]	
0052EEF5	50	PUSH EAX	OutlookE.0052EFDC
0052EEF6	8D55 EC	LEA EDX,DWORD PTR SS:[EBP-14]	
0052EEF9	8B83 1C030000	MOV EAX,DWORD PTR DS:[EBX+31C]	
0052EEFF	E8 E0AAF3FF	CALL OutlookE.004699E4	
0052EF04	8B55 EC	MOV EDX,DWORD PTR SS:[EBP-14]	
0052EF07	8B45 FC	MOV EAX,DWORD PTR SS:[EBP-4]	
0052EF0A	59	POP ECX	OutlookE.00435C37
0052EF0B	E8 7CF8FFFF	CALL OutlookE.0052E78C	
0052EF10	6A 00	PUSH 0	
0052EF12	66 8B0D 98EF5200	MOV CX,WORD PTR DS:[52EF98]	
0052EF19	B2 02	MOV DL,2	
0052EF1B	B8 A4EF5200	MOV EAX,OutlookE.0052EFA4	ASCII "Thank you for your register, pl
0052EF20	E8 F76CF0FF	CALL OutlookE.00435C1C	
0052EF25	C783 4C020000 01000000	MOV DWORD PTR DS:[EBX+24C],1	
0052EF2F	EB 23	JMP SHORT OutlookE.0052EF54	
0052EF31	6A 00	PUSH 0	
0052EF33	66 8B0D 98EF5200	MOV CX,WORD PTR DS:[52EF98]	
0052EF3A	B2 01	MOV DL,1	
0052EF3C	B8 DCEFF5200	MOV EAX,OutlookE.0052EFD0	ASCII "Sorry, your registration inform
0052EF41	E8 D66CF0FF	CALL OutlookE.00435C1C	

JE분기문에서 성공창과 실패창을 구분하고 ZF = 0일 때 점프 안하는 걸 볼 수 있다. 즉 AL != 0 이어야 한다.

JE를 NOP처리하면 성공 창이 뜨겠지만 이러면 완전히 Register된게 아니라서 이런 식으로 바꾸면 안된다.

0x0052EEDB으로 들어가서 분석을 해줘야한다.

- ZF (Zero Flag): 결과가 0이면 1, 0이 아니면 0

- TEST AL, AL: AL and AL

AL = 0일 경우 0 and 0 == 0이므로 ZF = 1

AL = 1일 경우 1 and 1 == 1이므로 ZF = 0

- JE (Jump if Equal) == Jump if Zero

ZF = 1 이면 점프, ZF = 0 이면 점프 안 함

0052EED7	8B45 FC	MOV EAX,DWORD PTR SS:[EBP-4]	
0052EEDA	59	POP ECX	
0052EEDB	E8 A4F7FFFF	CALL OutlookE.0052E684	
0052EEEE	84C0	TEST AL,AL	
0052EEEE2	74 4D	JE SHORT OutlookE.0052EF31	
0052EEEE4	8D55 F0	LEA EDX,DWORD PTR SS:[EBP-10]	

0x0052EEDB에 BP를 걸어준 후 다시 실행하면 해당 구간에서 멈추고 'F7'로 해당 함수 들어가서 살펴본다.

0052E5F8	- FF25 84F9B800	JMP DWORD PTR DS:[B8F984]
0052E5FE	8BC0	MOV EAX, EAX
0052E600	- FF25 80F9B800	JMP DWORD PTR DS:[B8F980]
0052E606	8BC0	MOV EAX, EAX
0052E608	55	PUSH EBP
0052E609	8BEC	MOV EBP, ESP
0052E60B	6A 00	PUSH 0
0052E60D	53	PUSH EBX
0052E60E	33C0	XOR EAX, EAX
0052E610	55	PUSH EBP
0052E611	68 54E65200	PUSH OutlookE.0052E652
0052E616	64:FF30	PUSH DWORD PTR FS:[EAX]
0052E619	64:8920	MOV DWORD PTR FS:[EAX], ESP
0052E61C	8055 FC	LEA EDI, OutlookE.0052E61C
0052E61F	B8 6CE65200	MOV EAX, OutlookE.0052E66C
0052E624	E8 C70CEFF	CALL OutlookE.0040F2F0
0052E629	8B45 FC	MOV EAX, DWORD PTR SS:[EBP-4]
0052E62C	BA 7CE65200	MOV EDI, OutlookE.0052E67C
0052E631	E8 9A64EDFF	CALL OutlookE.00404AD0
0052E636	75 04	JNZ SHORT OutlookE.0052E63C
0052E638	B3 01	MOV BL, 1
0052E63A	EB 02	JMP SHORT OutlookE.0052E63E
0052E63C	33D8	XOR EBX, EBX
0052E63E	33C0	XOR EAX, EAX
0052E640	5A	POP EDI
0052E641	59	POP ECX
0052E642	59	POP ECX
0052E643	64:8910	MOV DWORD PTR FS:[EAX], EDI
0052E646	68 5BE65200	PUSH OutlookE.0052E65B
0052E648	8045 FC	LEA EAX, DWORD PTR SS:[EBP-4]
0052E64E	E8 7160EDFF	CALL OutlookE.004046C4
0052E653	C3	RET
0052E654	^ F9 8759EDFF	JMP OutlookE.00403FF0

DS:[00B8F984]=01628947

We land on a far jump.
Press F8

해당 주소로 넘어오고 'F8'을 이용해서 넘어가면 JMP분기문이 있는 걸 볼 수 있다.
0x01628947로 넘어가는 걸 볼 수 있다.

01628947	6A 00	PUSH 0	
01628949	6A 00	PUSH 0	
0162894B	FF7424 10	PUSH DWORD PTR SS:[ESP+10]	OutlookE.0052E6E3
0162894F	FF7424 10	PUSH DWORD PTR SS:[ESP+10]	OutlookE.0052E6E3
01628953	E8 2BF0FFFF	CALL 01628783	
01628958	83C4 10	ADD ESP, 10	
0162895B	C2 0800	RET 8	
0162895E	6A 01	PUSH 1	
01628960	6A 00	PUSH 0	
01628962	FF7424 10	PUSH DWORD PTR SS:[ESP+10]	OutlookE.0052E6E3
01628966	FF7424 10	PUSH DWORD PTR SS:[ESP+10]	OutlookE.0052E6E3
0162896A	E8 14F0FFFF	CALL 01628783	
0162896F	83C4 10	ADD ESP, 10	

어디서 AL = 0이 되는지 찾기 위해 'F8'을 이용해서 차례로 실행해본다.

01628947	6A 00	PUSH 0	Registers (FPU) EAX 7FFDE000 ECX 0000083A EDX 7C90EB94 EBX 01F5D244
01628949	6A 00	PUSH 0	
0162894B	FF7424 10	PUSH DWORD PTR SS:[ESP+10]	
0162894F	FF7424 10	PUSH DWORD PTR SS:[ESP+10]	
01628953	E8 2BF0FFFF	CALL 01628783	
01628958	83C4 10	ADD ESP, 10	
0162895B	C2 0800	RET 8	

0x01628958에서 AL = 0이 되는 걸 볼 수 있다. BP를 걸고 재실행 후 F7'을 이용해서 해당 함수를 들어가본다.

016288C7	EB 62	JMP SHORT 0162892B
016288C9	57	PUSH EDI
016288CA	B3 01	MOV BL,1
016288CC	E8 94FEFEFF	CALL 01618765
016288D1	59	POP ECX
016288D2	FF75 0C	PUSH DWORD PTR SS:[EBP+C]
016288D5	8B0D B0EB6301	MOV ECX,DWORD PTR DS:[163EBB0]
016288DB	56	PUSH ESI
016288DC	E8 140BFFFF	CALL 016193F5
016288E1	84C0	TEST AL,AL
016288E3	75 04	JNZ SHORT 016288E9
016288E5	32DB	XOR BL,BL
016288E7	EB 15	JMP SHORT 016288FE
016288E9	807D 14 00	CMP BYTE PTR SS:[EBP+14],0
016288ED	74 0F	JE SHORT 016288FE
016288EF	FF75 0C	PUSH DWORD PTR SS:[EBP+C]
016288F2	8B0D B0EB6301	MOV ECX,DWORD PTR DS:[163EBB0]
016288F8	56	PUSH ESI
016288F9	E8 B700FFFF	CALL 016189B5
016288FE	E8 6CFEFEFF	CALL 0161876F
01628903	57	PUSH EDI
01628904	8BF0	MOV ESI,EAX
01628906	E8 86FCFFFF	CALL 01628591
0162890B	8B0D B0EB6301	MOV ECX,DWORD PTR DS:[163EBB0]
01628911	E8 1F0BFFFF	CALL 01619435
01628916	56	PUSH ESI
01628917	E8 49FEFEFF	CALL 01618765
0162891C	59	POP ECX
0162891D	E8 4DFEFEFF	CALL 0161876F
01628922	50	PUSH EAX
01628923	FF15 34F16201	CALL DWORD PTR DS:[162F134]
01628929	8AC3	MOV AL,BL
0162892B	5F	POP EDI
0162892C	5E	POP ESI
0162892D	5B	POP EBX
0162892E	C9	LEAVE
0162892F	C3	RETN

'F8'을 이용해서 내려가보면 RETN이 보이고 어디서 AL이 바뀌는 지 알 수 있다. 중간에 조건문이랑 CALL함수들이 많지만 마지막에 MOV AL, BL로 인해 AL이 변하고 RETN이 되는 걸 볼 수 있다. 즉, 위에 부분은 건들지 않고 해당 부분만 바꾸면 해결되는 걸 볼 수 있다.

* 중간의 분기문들 해석

1. "016288DC" 함수 실행

1.1 리턴 값 '0' ==> 분기되지 않음

1.2 리턴 값 '1' ==> "016288E9" 주소로 분기

2. 분기되지 않은 경우 (1-1 수행 됨)

2.1 "BL" 레지스터 값을 '0'으로 세팅

2.2 "016288FE" 주소로 분기

2.3 "016288FE" 주소에서 "0162876F" 함수 호출

3. 분기된 경우 (1-2 수행 됨)

3.1 "[EBX+14] 값을 '0'과 비교

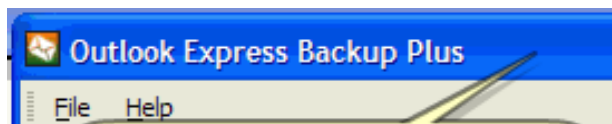
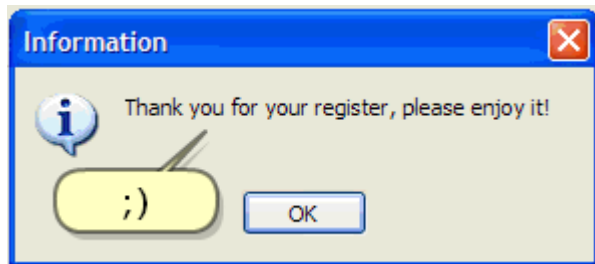
3.2 해당 값이 '0' ==> "0161876F" 주소로 분기

3.3 해당 값이 '0' 이 아님 ==> "016288 EF/F2/F8" 명령어 수행

3.3.1 "016288F9" 주소에서 "016289B5" 함수 호출

01628922	50	PUSH EHX
01628923	FF15 34F16201	CALL DWORD
01628929	B0 01	MOV AL, 1
0162892B	5F	POP EDI

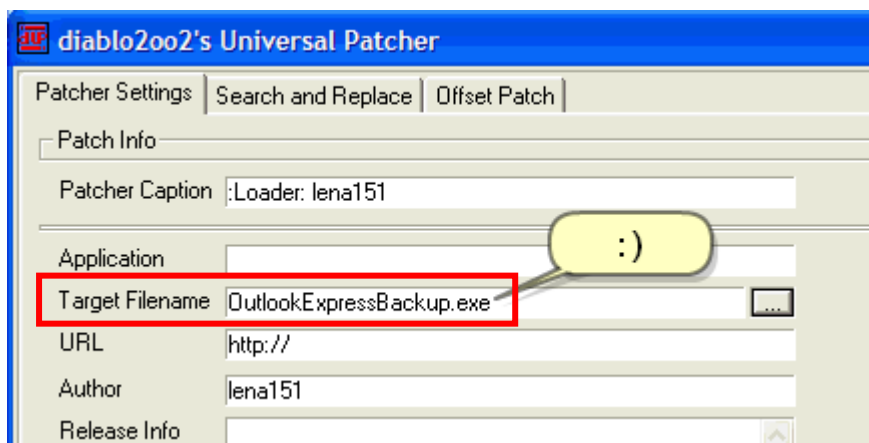
MOV AL, BL -> MOV AL, 1로 변경해주고 다시 실행해준다.



성공 창과 남은 시간이 없어진 걸 볼 수 있다.

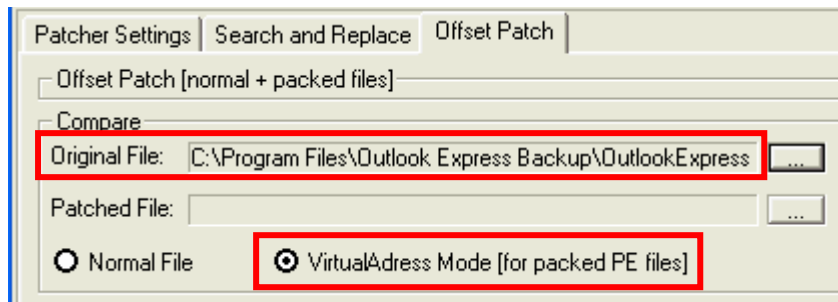
원래 OEP를 찾고 원본 코드에서 수정해줘야하는데 언패킹을 해주지 않고 수정을 했다. 그러면 패킹된 상태 이기 때문에 해당 파일을 다시 실행해보면 실행되지 않는다. (원래는 언패킹 => OEP 수정 => 임포트 테이블 수정)

이런 경우 로더를 이용해서 수정을 해줄거다. 하지만 여러 가지 이유 때문에 로더를 생성했던 시스템에서만 동작할 것이다. **가장 큰 이유는 프로그램이 실행될 때의 메모리 주소를 이용하여 패치하기 때문이다.** 실행되는 운영체제 환경마다 메모리 주소가 다르기 때문에 로더의 호환성을 높이려면 메인 프로그램의 코드 자체를 수정해야 한다.



"dUP" 이라는 로더 툴을 이용해서 수정해줄거다.

"Target Filename"에 수정해야 할 파일(OutlookExpressBackup.exe)을 선택해준다.



Offset Patch로 넘어가서 VirtualAddress Mode에 체크해주고 "Original File"에 원래 파일 (OutlookExpressBackup.exe)을 넣어준다.

01628923	FF15 34F16201	CALL DWORD PTR DS:[162F134]
01628929	8AC3	MOV AL,BL
0162892B	5F	POP EDI

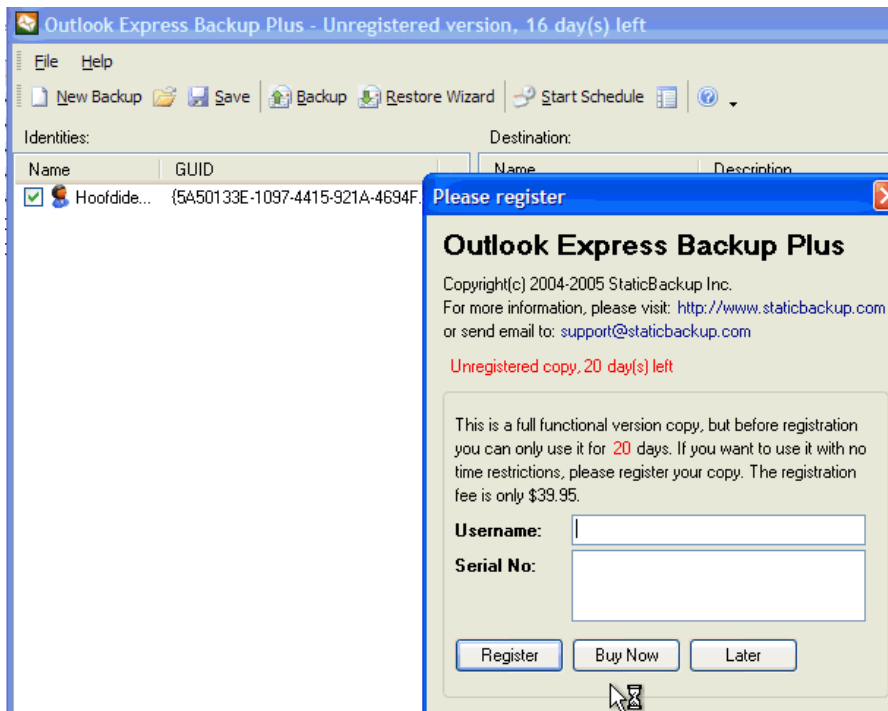
01628922	50	PUSH EHX
01628923	FF15 34F16201	CALL DWORD PTR DS:[162F134]
01628929	B0 01	MOV AL,1
0162892B	5F	POP EDI

Add Bytes
Offset: 1628929
Original Byte: 8A
Patched Byte: B0
Add

Add Bytes
Offset: 162892A
Original Byte: C3
Patched Byte: 01
Add

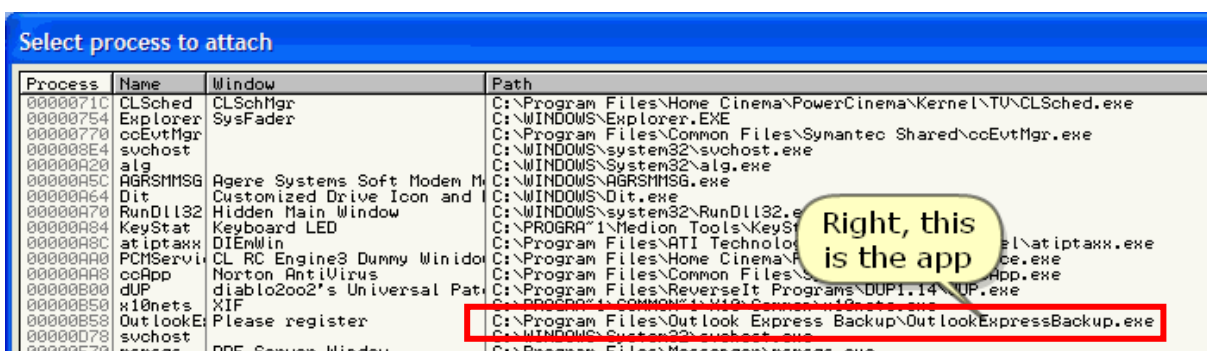
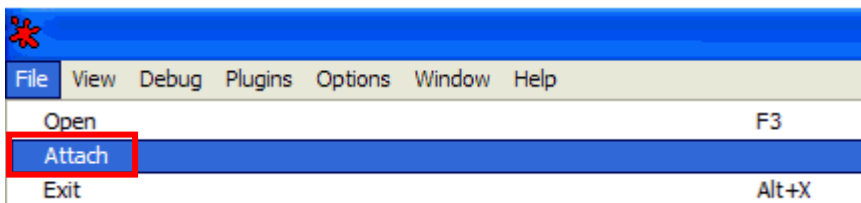
VA Offset	Original Byte	Patched Byte
1628929	8A	B0
162892A	C3	01

아까 수정한 부분을 1Byte씩 넣어서 수정해준다. 수정해 준 후 "Crate Loder"을 한 후 파일을 생성해준다.

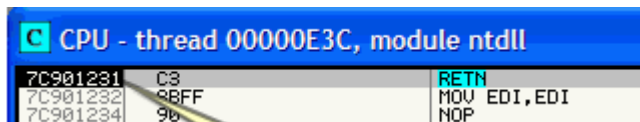


하지만 실행해보면 수정이 안 된 걸 볼 수 있다.

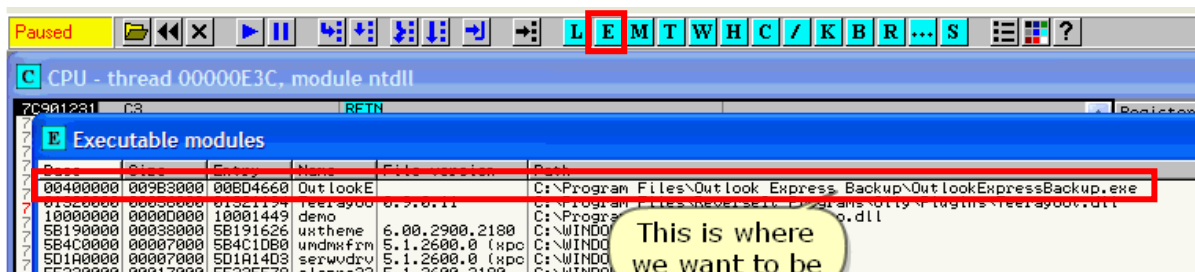
왜냐하면 "Armadillo" 프로텍터는 로더가 메모리 블록에 어떤 값을 작성하고자 하는 것을 탐지하여 "VA" 주소를 변경해버린다. 따라서 로더는 메인 프로그램을 패치할 수 없게 된다. 디버거에서 분석을 통해 어떤 부분이 문제고 어떻게 패치 하는지 설명하도록 하겠다.



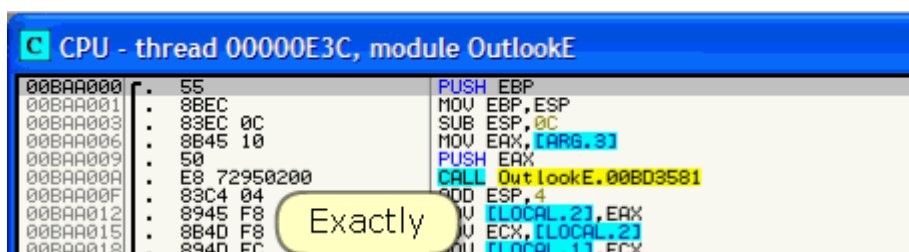
로더를 실행 시킨 후 올리디버그를 돌아가서 Attach를 해 준 후 OutlookExpressBackup.exe를 Attach해준다.



아직 진짜 프로그램 코드로 들어가기 전에 있다.



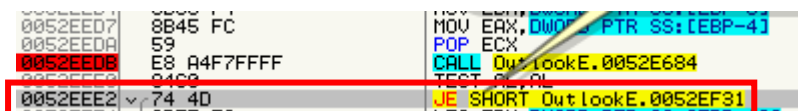
Executable modules로 들어가서 박스부분을 눌러준다.



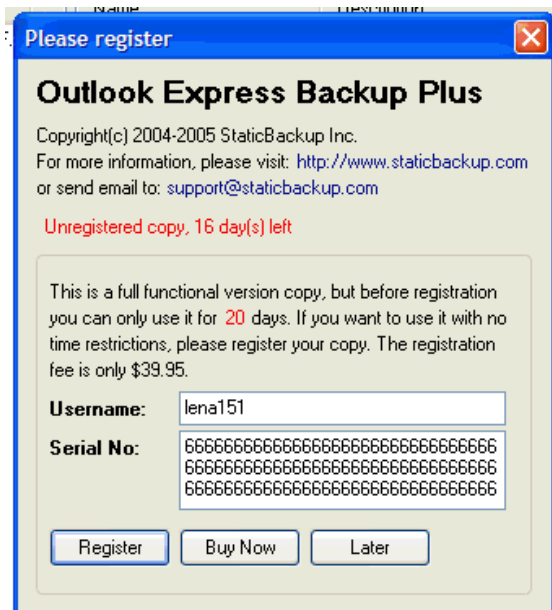
그럼 여기로 넘어가는 걸 볼 수 있다.



'F9'를 이용해서 실행해주면 레지스터 값이 아무것도 안보이는데 올리디버그가 혼란에 빠져서 그런거다 이 부분은 신경쓰지 않아도 된다.



처음 JE분기문이 있는 곳으로 넘어온다. (성공 창과 실패 창을 고르는 곳이 있는 곳)



다시 값들을 입력해주고 "Register"을 눌러준다.

0052EED8	59	POP ECX
0052EED9	E8 A4F7FFFF	CALL OutlookE.0052E684
0052EEE0	84C0	TEST AL,AL
0052EEE2	74 4D	JE SHORT OutlookE.0052EF31
0052EEE3	8BFF FA	MOV EBX,EBP

BP 구간에서 멈추는 걸 볼 수 있다.

0052E5F8	- FF25 84F9B800	JMP DWORD PTR DS:[B8F984]
0052E5FE	8BC0	MOV EAX,EAX

014A8947	6A 00	PUSH 0
014A8949	6A 00	PUSH 0
014A894B	FF7424 10	PUSH DWORD PTR SS:[ESP+10]
014A894F	FF7424 10	PUSH DWORD PTR SS:[ESP+10]
014A8953	E8 2BFFFFFF	CALL 014A8783
014A8958	83C4 10	ADD ESP,10

(현재)

01628947	6A 00	PUSH 0
01628949	6A 00	PUSH 0
0162894B	FF7424 10	PUSH DWORD PTR SS:[ESP+10]
0162894F	FF7424 10	PUSH DWORD PTR SS:[ESP+10]
01628953	E8 2BFFFFFF	CALL 01628783
01628958	83C4 10	ADD ESP,10

(전)

아까처럼 계속 넘어가보면 아까와 같은 0x0052E5F8에 도달하는 걸 볼 수 있다. 'F8'을 이용해서 넘어가본다. 근데 아까와 다르게 0x014A8947로 넘어가는 걸 볼 수 있다. (즉 주소 값이 달라진 걸 볼 수 있다.).

CALL이 부르는 함수 주소도 변경된 걸 볼 수 있다.

014A894B	FF7424 10	PUSH DWORD PTR
014A894F	FF7424 10	PUSH DWORD PTR
014A8953	E8 2BFFFFFF	CALL 014A8783
014A8958	83C4 10	ADD ESP,10
014A895C	C3 0000	RETN 0

해당 부분에 BP를 걸고 다시 실행해서 해당 함수에 들어가본다.

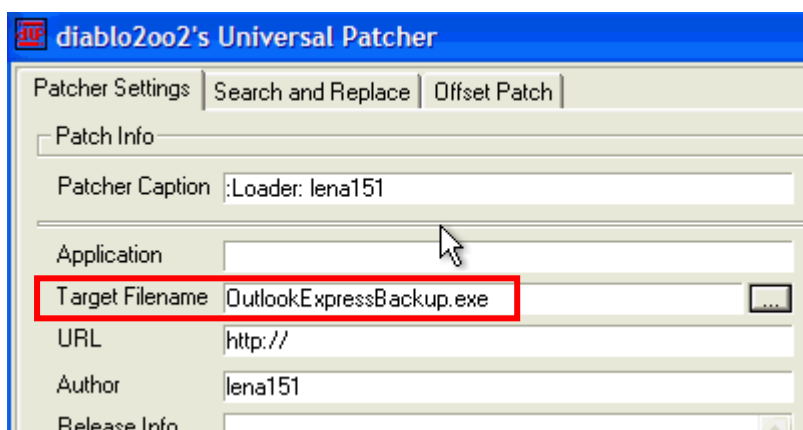
014A8917	50	49FEFEFF	PUSH ESI
014A891C	59		CALL 01498765
014A891D	58	40FEFEFF	POP ECX
014A8922	50		CALL 0149876F
014A8923	50		PUSH EAX
014A8928	FF15 34F14001		CALL DWORD PTR DS:[14AF134]
014A8929	8AC3		MOV AL,BL
014A892B	5F		POP EDI
014A892C	5E		POP ESI

(현재)

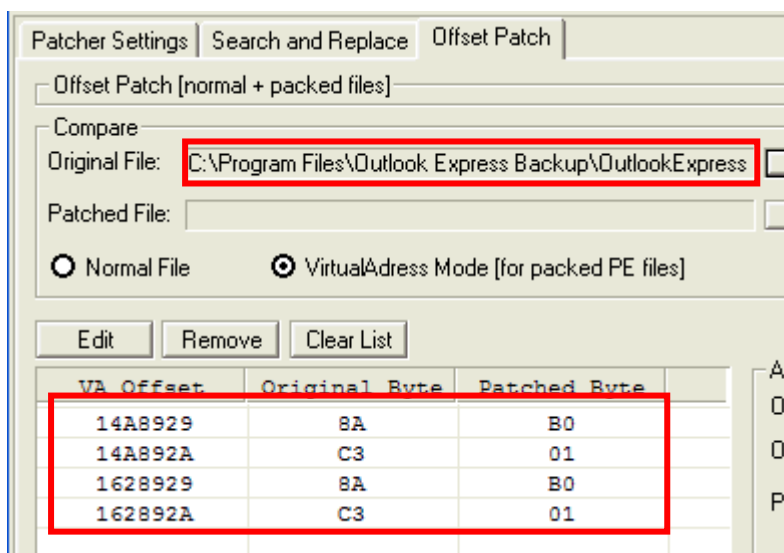
01628922	50		PUSH EAX
01628923	FF15 34F16201		CALL DWORD PTR DS:[162F134]
01628929	8AC3		MOV AL,BL
0162892B	5F		POP EDI
0162892C	5E		POP ESI

(전)

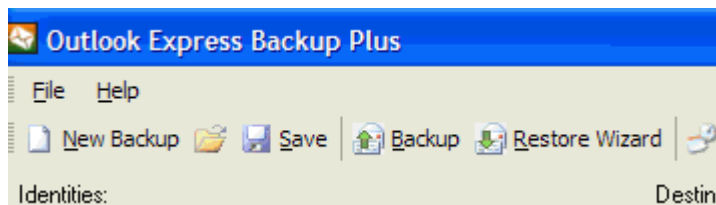
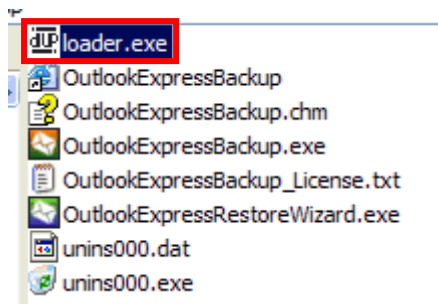
쭉 실행하다보면 아까와 주소가 다른 걸 볼 수 있다.



다시 “dUP” 로더를 이용해서 수정해줄거다. Target Filename에 수정할 파일을 넣어준다.(attach했기 때문에 해당 파일로 넣어주면 된다.)



로더 실행 시 변경된 VA 주소값과 이전과 동일한 주소 값을 패치해주면 된다.



로더를 실행해주면 모든 값이 변한 걸 볼 수 있다.

-> 레나가 첫 번째 로더의 이름을 loder.exe로 저장했는데 attach한 것은 OutlookExpressBackup.exe 이걸로 불러온 걸 볼 수 있다. 근데 왜 loder.exe가 아니라 OutlookExpressBackup.exe 이걸 실행했지,,? 그리고 두 번째 로더를 만들기 위해서 왜 Target Filename과 Original File을 둘 다 OutlookExpressBackup.exe 이걸로 했는지 잘 모르겠다,,, 보기 편하게 원래의 이름으로 바꾼건가,,

다른 사람이 작성한 설명을 보면

우선 로더를 통해 해당 바이너리를 실행한 상태에서 디버거에 "Attach" 시킨다. 그러면 디버거는 "Attach" 된 바이너리의 메모리 주소 값을 볼 수 있게 된다. 쪽쪽 분석하다 보면 우리가 "MOV AL,BL => MOV AL,1" 로 패치한 주소를 포함하는 코드 루틴으로 이동 될 것이다. 그런데 실행되는 코드는 같아도 주소가 다를 것이다. 즉 프로텍터가 로더 실행을 탐지하고 "VA" 주소를 변경하였기 때문에 절대 주소를 이용하는 기존의 로더는 무용지물이 된 것이다. 패치하는 방법은 의외로 간단한데 바로 로더에서 OPCODE 패치를 4 번하면 된다.

이런 식으로 설명되어 있는데 직접 실행해 볼 수 없어서 레나가 왜 저렇게 했는지 확인이 불가능하다,,,